ΜΙΑ ΠΡΩΤΗ ΑΡΧΙΚΗ ΓΝΩΡΙΜΙΑ ΜΕ ΤΗΝ ΙΔΙΑΙΤΕΡΗ «ΑΡΓΚΟ» ΤΗΣ PYTHON

ΓΙΑ ΤΗΝ ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΕΦΕΙΑ

Μ.Ι.Σ. 2025-2026

Εισαγωγικές Παρατηρήσεις

Η σχεδίαση ενός λογισμικού συστήματος μπορεί να υλοποιηθεί με διάφορες μεθοδολογίες ανάπτυξης. Οι μεθοδολογίες αυτές κατηγοριοποιούνται συνήθως, σε διαδικασιακές και σε διαδικασίες που είναι προσανατολισμένες προς την έννοια των αντικειμένων. Αυτή η δεύτερη γενική κατηγοριοποίηση που ονομάζεται αντικειμενοστρέφεια (object-oriented) αφορά και την PYTHON.

Η ακολουθιακή ή διαδικασιακή προσέγγιση προγραμματισμού και η δομημένη ανάλυση, παρουσιάστηκε αναλυτικά στις πρώτες διαφάνειες. Αυτό το παρεχόμενο εκπαιδευτικό υλικό περιελάμβανε όλα τα απαραίτητα βασικά γνωσιακά στοιχεία για να δημιουργήσει στον εκπαιδευόμενο, το κατάλληλο, θεωρητικό και πρακτικό υπόβαθρο γνώσης για τον διαδικασιακό τρόπο προγραμματισμού.

Στις διαφάνειες που ακολουθούν οι εκπαιδευόμενοι εισάγονται σε μια διαφορετική τεχνοτροπία ανάπτυξης και προγραμματισμού που ονομάζεται, αντικειμενοστραφής σχεδίαση και αντικειμενοστραφής τεχνολογία προγραμματισμού.

Η αντικειμενοστραφής προσέγγιση προγραμματισμού περιλαμβάνει ένα εκτεταμένο πεδίο γνώσης.

Ο όρος, αντικειμενοστραφής προγραμματισμός, προέρχεται από τον αγγλικό όρο, Object Oriented Programming (O.O.P.), που στα ελληνικά θα μπορούσε να μεταφραστεί απλοποιημένα, από έναν Πληροφορικό, σαν «προγραμματισμός, προσανατολισμένος γύρω από τα αντικείμενα». Και βέβαια η παράθεση της συγκεκριμένης έκφρασης, από μόνη της δεν επαρκεί, καθώς δεν γίνεται κατανοητή ειδικά από τους νέους προγραμματιστές και θα πρέπει να επεξηγηθεί, μέσω κάποιας εξοικείωσης από προϋπάρχουσα γνώση και εμπειρία από τον κλασικό διαδικασιακό προγραμματισμό.

Ο όρος, αντικειμενοστραφής, εμφανίζεται στην ελληνική βιβλιογραφία και σαν αντικειμενοστρεφής, με την ίδια ακριβώς σημασία.

Μ.Ι.Σ. 2025-2026

Η έκφραση, Object Oriented Programming, στην πραγματικότητα σημαίνει, μια φιλοσοφία αντίληψης του προγραμματισμού, με ένα διαφορετικό πνεύμα, ή παρατήρησης μέσα από ένα διαφορετικό πρίσμα. Για παράδειγμα, αυτό που σε μια άλλη τεχνοτροπία θα ονομαζόταν ρουτίνα, στην Ο.Ο.Ρ. φιλοσοφία, θα ονομάζεται, μέθοδος (method). Ο αντικειμενοστραφής προγραμματισμός, δεν είναι μια νέα τεχνοτροπία, όπως λανθασμένα προβάλλεται, αλλά υπάρχει και εξελίσσεται σαν παράλληλη τεχνοτροπία προγραμματισμού ήδη από την δεκαετία του 1980, με ρίζες από την δεκαετία του 1960.

Συνεπώς ο αντικειμενοστραφής προγραμματισμός δεν είναι σε καμία περίπτωση το εκπληκτικό νέο περιβάλλον ανάπτυξης, αλλά ένα διαθέσιμο επιπλέον εργαλείο στον τομέα της Τεχνολογίας Λογισμικού, της επιστήμης της Πληροφορικής.

Δανειζόμενοι την επεξήγηση του όρου, από ένα από τα σημαντικότερα πρόσωπα του χώρου, τον Grady Booch, το Object Oriented Programming είναι, η μεθοδολογία να συγγράφουμε κώδικα (συναρτήσεις, ρουτίνες, ή μεθόδους), με τέτοιο τρόπο ώστε, να συνδυάζουμε λειτουργικές εντολές προγραμματισμού (ή διαδικασίες) αλλά και δεδομένα (ή πληροφορίες) και όταν το πετυχαίνουμε αυτό, να αποθηκεύουμε αυτή την κατασκευή, σε μια δομική μορφή, που θα την ονομάζουμε object, (δηλαδή, αντικείμενο). Τα αντικείμενα αυτά μπορούμε επιπλέον να τα τακτοποιούμε σε συλλογές αντικειμένων, εφαρμόζοντας μια ιεραρχική δομή σχέσεων όπου η κάθε συλλογή θα μπορεί να κληρονομεί (ή να κληρονομείται) λειτουργικότητα, προς (ή από) μια άλλη συλλογή αντικειμένων. Και από αυτόν τον ορισμό προήλθε η έκφραση αντικειμενοστραφής προγραμματισμός, που σε απλά ελληνικά θα μπορούσε να σημαίνει, «***φτιάξε κώδικα που χρησιμοποιεί κατάλληλες διεπαφές με άλλους κώδικες, επιτελεί μια συγκεκριμένη λειτουργία, μετασχηματίζει δεδομένα και φύλαξε τον να υπάρχει και όταν τον χρειαστείς κάλεσε τον»***.

Η αντικειμενοστρέφεια, όπως απλοποιημένα αποκαλείται, από τους προγραμματιστές θεωρείται, για ένα λογισμικό, μια συλλογή από αντικείμενα (object), το καθένα από τα οποία περιέχει διαδικασίες και δεδομένα. Η κλήση ενός αντικειμένου, από ένα άλλο αντικείμενο επιστρέφει στο αντικείμενο που έκανε την κλήση, ένα είδος «υπηρεσιών» με την μορφή δεδομένων αλλά και κώδικα. Αυτή η διαδοχική παροχή κατώτερων «υπηρεσιών» προς άλλες «υπηρεσίες» ανώτερου επιπέδου, δημιουργεί τελικά, την αποτελεσματικότητα μιας εφαρμογής προς ένα εξωτερικό περιβάλλον (τους χρήστες).

Μ.Ι.Σ. 2025-2026

Η αντικειμενοστρέφεια, εισάγει πολλές έννοιες ιδιαίτερης σημασιολογίας και ειδικές προγραμματιστικές τεχνικές που δημιουργούν σύγχυση και αποπροσανατολίζουν.

Αυτά τα προβλήματα της αντικειμενοστρέφειας διαφαίνεται να επιλύονται διαμέσου του τομέα της τεχνολογίας λογισμικού και των συστημάτων σχεδιασμού U.M.L. (Unified Modeling Language), της γλώσσας που επίσης δημιουργήθηκε από τον Grady Booch, του προσώπου από το οποίο δανειστήκαμε την επεξήγηση για τον όρο, Ο.Ο.Ρ.

Η U.M.L. είναι μια τυποποιημένη «γλώσσα» που περιγράφει τα συστήματα λογισμικού με ειδικά σχεδιαγράμματα και βασίζεται στις αρχές και τα πρότυπα της αντικειμενοστρέφειας.

Η U.M.L. ως παγκόσμιο πρότυπο παράστασης λογισμικού, επιτρέπει την οπτική αναπαράσταση, της ανάλυσης, της καταγραφής των απαιτήσεων, της σχεδίασης, της υλοποίησης και της τεκμηρίωσης ενός πληροφοριακού συστήματος, με τρόπο κατανοητό στον καθένα που εμπλέκεται στην ανάπτυξη, χρήση και υποστήριξη ενός λογισμικού (πελάτη, αναλυτή, μηχανικό λογισμικού, προγραμματιστή, χειριστή, κ.λπ.).

Μ.Ι.Σ. 2025-2026

Αντικείμενα – Object

Ο όρος αντικείμενο (object), όπως εισαγωγικά επεξηγήθηκε αφορά την περιγραφή μιας δομικής μορφής κώδικα, η οποία βασίζεται σε δεδομένα (ή «στοχεύει» τα δεδομένα), τα οποία μέσω κάποιων λειτουργικών εντολών προγραμματισμού (διαδικασίες ή υπολογισμούς) τα μετασχηματίζει σε μετα-δεδομένα, την χρονική στιγμή που θα κληθεί.

Στην αντικειμενοστρέφεια, ένα object, θα ανταποκριθεί μια συγκεκριμένη χρονική στιγμή σε μια κατάλληλη κλήση, από ένα άλλο κατάλληλο αντικείμενο (object), το οποίο και αυτό μπορεί να έχει κληθεί με την σειρά του από ένα άλλο ανώτερο object.

Αντικείμενο (object), μπορεί να χαρακτηριστεί και μια συλλογή δεδομένων, ακόμα και πρωτογενούς τύπου, που μετασχηματίζεται ή δεν μετασχηματίζεται και απλά χρησιμοποιείται.

Για την εκπαίδευση της πληροφορικής, η δυσκολία κατανόησης του όρου object, θεωρείται δεδομένη και απαιτεί χρόνο και εμπειρίες. Μια κατάλληλη όμως σχηματική απεικόνιση μπορεί να λειτουργήσει υποβοηθητικά.

Η ανωτέρω εικόνα αποτυπώνει -αφαιρετικά- για την αντικειμενοστρέφεια, τις έννοιες των objects και της ενθυλάκωσης (ecapsulation) και διαφοροποιείται εννοιολογικά με την αντίστοιχη απεικόνιση στον ακολουθιακό προγραμματισμό.

Μ.Ι.Σ. 2025-2026

Κλάσεις – Class

Ο όρος κλάση (class), προέρχεται από την αγγλική συντομογραφία class, του όρου classification, το οποίο μπορεί να αποδοθεί καλύτερα, όχι στην ελληνική λέξη «κλάση», ούτε στην ελληνική λέξη «ταξινόμηση» αλλά στις ελληνικές λέξεις, «κατάταξη», «ένταξη» ή «**τακτοποίηση**».

Ο όρος κλάση, δημιουργεί μια από τις μεγαλύτερες συγχύσεις στην αντικειμενοστρέφεια καθώς δεν περιγράφει την «ταξινόμηση των πραγμάτων» αλλά την «**τακτοποίηση των πραγμάτων**» σε τάξεις (σε κατηγορίες). Συγκεκριμένα, με την είσοδο μας σε μια αίθουσα διδασκαλίας, μπορούμε να παρατηρήσουμε, «τάξεις πραγμάτων», όπως, οι σπουδαστές, τα έπιπλα, τα βιβλία, τα μαθήματα, τα εποπτικά μέσα αλλά και κάθε άλλο αντικείμενο (υλικό ή άυλο) για το οποίο θα του άξιζε η προσοχή μας.

Συνεπώς, εάν μας ζητείτο να κατασκευάσουμε ένα λογισμικό σε μια αντικειμενοστραφή γλώσσα, που να υποστηρίζει την λειτουργία των εκπαιδευτικών δομών της ΔΕΛΤΑ, θα χρειαζόταν οπωσδήποτε να δημιουργήσουμε τις κλάσεις, ΣΠΟΥΔΑΣΤΕΣ, ΕΠΙΠΛΑ, ΒΙΒΛΙΑ, ΜΑΘΗΜΑΤΑ και ΕΠΟΠΤΙΚΑ ΜΕΣΑ. Ακολούθως, θα μπορούσαμε να κατατάξουμε (και όχι να ταξινομήσουμε) τα object, ΤΗΛΕΟΡΑΣΗ, ΠΙΝΑΚΑΣ και ΠΟΛΥΜΕΤΡΟ στην κλάση των ΕΠΟΠΤΙΚΩΝ ΜΕΣΩΝ. Τον σπουδαστή ΕΛΕΥΘΕΡΙΟΥ και τις σπουδάστριες ΝΙΚΟΥ και ΔΗΜΟΥ στην κλάση ΣΠΟΥΔΑΣΤΕΣ, αλλά να δημιουργήσουμε και μια άλλη class για τα άυλα object όπως η ΠΛΗΡΟΦΟΡΙΚΗ, η ΦΥΣΙΚΗ και η ΗΛΕΚΤΡΟΛΟΓΙΑ που ανήκουν φυσικά στην κλάση ΜΑΘΗΜΑΤΑ.

Όμως η αντικειμενοστρέφεια θα μας επέτρεπε να δημιουργήσουμε και μια άλλη κλάση, την οποία να μπορούσαμε να ονομάσουμε ΣΥΝΑΡΤΗΣΕΙΣ-ΕΛΕΓΧΟΥ και μέσα της να τοποθετήσουμε σαν αντικείμενα, σχετικές μεθόδους (και όχι αντικείμενα του φυσικού κόσμου) που διαχειρίζονται γενικότερα στο λογισμικό μας, ημερομηνίες και ώρες μαθημάτων. Οι κλάσεις συνεπώς περιέχουν και όλες τις απαραίτητες μεθόδους που έχουν σχέση με την κλάση και τα στοιχεία της.

Μ.Σ. 2025-2026

Στιγμιότυπα – Instances

Έχοντας περιγράψει τον βασικό όρο της κλάσης και χρησιμοποιώντας για παράδειγμα την κατασκευή ενός λογισμικού με τις κλάσεις ΣΠΟΥΔΑΣΤΕΣ, ΕΠΙΠΛΑ, ΒΙΒΛΙΑ, ΜΑΘΗΜΑΤΑ, ΕΠΟΠΤΙΚΑ ΜΕΣΑ και ΣΥΝΑΡΤΗΣΕΙΣ-ΕΛΕΓΧΟΥ, θα συναντήσουμε στην αντικειμενοστρέφεια και τον βασικό όρο στιγμιότυπο (instance).

Ο όρος αφορά τα αντικείμενα, ΤΗΛΕΟΡΑΣΗ, ΕΛΕΥΘΕΡΙΟΥ, ΠΛΗΡΟΦΟΡΙΚΗ αλλά και όλα τα υπόλοιπα όμοια αντικείμενα, τα οποία, το καθένα είναι ένα στιγμιότυπο της κλάσης στην οποία ανήκει. Επιπλέον, ένα στιγμιότυπο, μπορεί να διαφοροποιείται ανάλογα με το περιβάλλον που μπορεί να το επηρεάζει. Για παράδειγμα, το μάθημα ΠΛΗΡΟΦΟΡΙΚΗ, το οποίο διδάσκεται σήμερα στις 14.00, εάν το παρατηρήσουμε στις 11.00 το πρωί είναι ένα στιγμιότυπο Α, ενός μαθήματος που δεν έχει πραγματοποιηθεί, αλλά στις 18.00 το απόγευμα, είναι πάλι το ίδιο μάθημα (το ίδιο αντικείμενο) που έχει όμως πραγματοποιηθεί και άρα αποτυπώνεται, αφού επηρεάζεται διαφορετικά από το περιβάλλον του (τον χρόνο) σαν ένα άλλο στιγμιότυπο Β, του ίδιου αντικειμένου (του μαθήματος ΠΛΗΡΟΦΟΡΙΚΗΣ) της κλάσης ΜΑΘΗΜΑΤΑ.

Μέθοδοι – Methods

Με τον όρο μέθοδος (method), περιγράφουμε δομικά στοιχεία κώδικα ή ποιο απλά, σειρά από εντολές προγραμματισμού. Την ίδια ακριβώς σημασία αποτυπώνουν οι γνωστοί όροι, ρουτίνα, υπορουτίνα και συνάρτηση. Μέσα σε μια method, οι εντολές προγραμματισμού εκτελούνται ακολουθιακά, με την γνωστή λογική της «μπίλιας» και των πιθανών εντολών ανακατεύθυνσης. Μια μέθοδος μπορεί να κληθεί από μια άλλη μέθοδο ή να καλεί με την σειρά της άλλες μεθόδους. Και στην περίπτωση της αντικειμενοστρέφειας έχουμε την κυρίαρχη, αρχική main μέθοδο. Οι μέθοδοι συνοδεύονται με τις κλασικές έννοιες, των ορισμάτων εισόδου, των ορισμάτων εξόδου, των επιστροφών, αλλά επίσης και με νέες έννοιες, όπως, κατασκευάστρια μέθοδος, μέθοδος getter, μέθοδος setter, κ.λπ.

Μ.Υ. 2025-2026

Κληρονομικότητα – Inheritance

Ο όρος κληρονομικότητα (inheritance), είναι ίσως η σπουδαιότερη τεχνική ευκολία της τεχνοτροπίας Ο.Ο.Ρ. Η κληρονομικότητα βασίζεται στην σχέση γενίκευσης – εξειδίκευσης μεταξύ κλάσεων και συνδέεται άμεσα με τον όρο υπό-κλάσεις (subclass). Αυτό που γίνεται με την κληρονομικότητα στην πραγματικότητα είναι ότι, οι κλάσεις διασυνδέονται μεταξύ τους, με έναν κατάλληλο τρόπο, ώστε δομικά στοιχεία τα οποία μπορεί να είναι κοινά, να μετακινούνται σε κλάσεις «απογόνους» και να αποφεύγεται η επανάληψη όμοιου κώδικα, όμοιων μεταβλητών και όμοιων συστατικών λογισμικών (συναρτήσεων ή μεθόδων για την Ο.Ο.Ρ.). Αυτό μπορεί να είναι ιδιαίτερα ωφέλιμο καθώς απλοποιεί την φάση της ανάπτυξης ενός αξιόλογου λογισμικού αλλά μπορεί να είναι πολύ περισσότερο ωφέλιμο κατά την συνεχή φάση της συντήρηση του.

Είναι ευκολότερο να συντηρήσεις μια εφαρμογή με 50 κλάσεις, από μια εφαρμογή με 80 κλάσεις όπου πολλά πράγματα επαναλαμβάνονται.

Για να γίνει αυτό κατανοητό και στο επίπεδο της δεδομένης απειρίας των εκπαιδευομένων στην Ο.Ο.Ρ. αλλά και της ανάγκης να λάβουν ένα ικανό γνωσιακό υπόβαθρο για την περαιτέρω μελέτη του θέματος της κληρονομικότητας θα χρησιμοποιήσουμε για παράδειγμα μια εφαρμογή διαχείρισης των ΕΛ.ΤΑ., τα οποία μας εξυπηρετούν με τις ταχυδρομικές τους υπηρεσίες. Σε αυτό το σύστημα θα μπορούσαν να υπάρχουν πολλές δεκάδες κλάσεις αλλά εμείς θα επικεντρωθούμε σε κάποιες κλάσεις οι οποίες είναι αναγκαίες για την διαχείριση των χάρτινων κιβωτίων (πακέτων), σε ότι αφορά την καταμέτρηση του όγκου τους, έτσι ώστε να μπορούν τα ΕΛ.ΤΑ. να υπολογίζουν τις αναγκαίες και διαθέσιμες παλέτες φόρτωσης.

Ας υποθέσουμε τώρα ότι τα χάρτινα κιβώτια τα οποία αναλαμβάνουν τα ΕΛ.ΤΑ. να διαχειριστούν, διαχωρίζονται σε δυο βασικές μορφές, α) τα ορθογώνια κουτιά και β) τα κυλινδρικά κουτιά ή κυλίνδρους. Στην περίπτωση αυτή μπορούμε να υποθέσουμε ότι θα χρειαστούμε δυο διαφορετικές κλάσεις, μια για τα κιβώτια και μια για τους κυλίνδρους.

Μ.Ι.Σ. 2025-2026

Εφόσον προσπαθούμε να αναπτύξουμε μια εφαρμογή για την διαχείριση αυτών των αντικειμένων και για να καταφέρουμε να υπολογίζουμε τους όγκους τους, θα χρειαζόμασταν κατά περίπτωση,

α) για τα κιβώτια,

α.1) το πλάτος,
α.2) το μήκος,
α.3) το ύψος,

β) για τους κυλίνδρους,

β.1) το ύψος,
β.2) την ακτίνα,

και για την ολοκλήρωση των δυο κλάσεων, επίσης,

γ.1) την περιγραφή του κουτιού,
γ.2) το χρώμα του κουτιού,
γ.3) το υλικό του κουτιού.

Στην περίπτωση του υπολογισμού του όγκου των κιβωτίων ο αλγόριθμος αποτυπώνεται με την πράξη, πλάτος x μήκος x ύψος, ενώ για τους κυλίνδρους, με την πράξη, ακτίνα^2 x π x ύψος.

Συνεπώς, οι δύο κλάσεις, για την διαχείριση των αντικειμένων αυτών, μαζί με τα επιπλέον περιγραφικά τους στοιχεία (περιγραφή, χρώμα, κ.λπ.) θα αποτυπώνονταν στην σχεδίαση μας ως εξής,

Μ.Ι.Σ. 2025-2026

Ορθογώνια Κουτιά
πλάτος
μήκος
ύψος
περιγραφή
χρώμα
υλικό

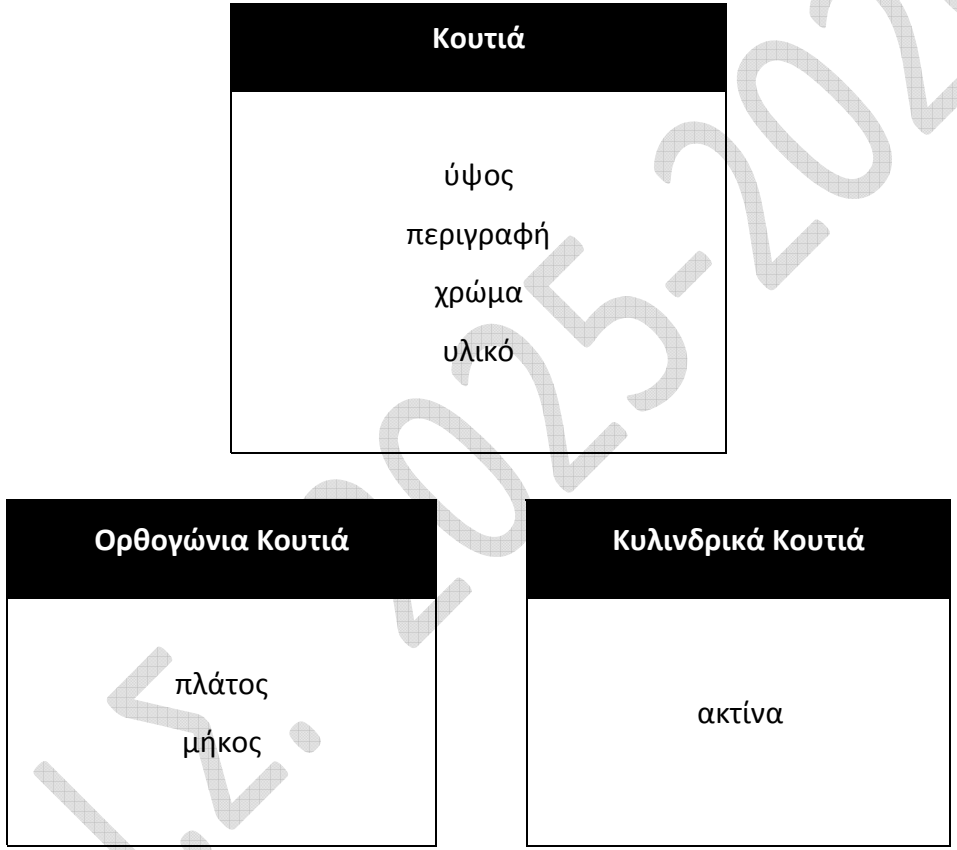
Κυλινδρικά Κουτιά
ύψος,
ακτίνα
περιγραφή
χρώμα
υλικό

Από την αντιπαράθεση των δυο αυτών κλάσεων παρατηρούμε ότι, υπάρχουν στοιχεία που επαναλαμβάνονται και στοιχεία που δεν επαναλαμβάνονται. Αυτό θα μπορούσε να σημαίνει ενδεχομένως ότι στην συνέχεια της ανάπτυξης του λογισμικού, θα είμαστε αναγκασμένοι να δημιουργούμε μεθόδους για όμοιες λειτουργικότητες. Για παράδειγμα, έναν έλεγχο ορθότητας, ώστε να μην δέχονται τα ΕΛ.ΤΑ. για φόρτωση, υπερμεγέθη κιβώτια σε ύψος, ή έναν έλεγχο για την αποτροπή αποδοχής κουτιού χωρίς διαστάσεις, ή χωρίς τύπο υλικού. Θα έπρεπε δηλαδή να κατασκευάσουμε μια μέθοδο για να υπολογίζει ένα ανώτατο όριο ύψος στα κουτιά και να βγάζει ένα ενημερωτικό μήνυμα αλλά και μια παρόμοια μέθοδο για να υπολογίζει επίσης το ανώτατο όριο ύψος στους κυλίνδρους. Και φυσικά αυτό θα μπορούσε να αφορά και πολλές άλλες μεθόδους ελέγχου, για το υλικό, τις ώρες παραλαβής, το ανώτατο βάρος, κ.λπ.

Αυτή η περιγραφή των καταστάσεων ελέγχου αναδεικνύει το πρόβλημα της συγγραφής κώδικα που στην πραγματικότητα θα εκτελεί τα ίδια πράγματα. Μήπως λοιπόν για να το αποφύγουμε αυτό, θα ήταν καλύτερα να βγάλουμε τα κοινά στοιχεία, σε μια κοινή κλάση και τα ειδικά στοιχεία να τα αφήσουμε σε δυο υποκλάσεις για την κάθε περίπτωση κουτιού. Συνεπώς, οι δύο αρχικές κλάσεις ΟΡΘΟΓΩΝΙΑ ΚΟΥΤΙΑ και ΚΥΛΙΝΔΡΙΚΑ ΚΟΥΤΙΑ, μπορούν να διασπαστούν σε τρεις κλάσεις και στην νέα Ο.Ο.Ρ., σχεδίαση μας να αποτυπωθούν ως, ΚΟΥΤΙΑ, ΟΡΘΟΓΩΝΙΑ ΚΟΥΤΙΑ και ΚΥΛΙΝΔΡΙΚΑ ΚΟΥΤΙΑ.

Μ.Σ. 2025-2026

Αυτή ακριβώς είναι η ιδέα, τα στοιχεία που είναι κοινά να τοποθετούνται σε μια υπέρ-κλάση (superclass) και τα διαφορετικά στοιχεία να παραμένουν σε υπό-κλάσεις (subclass), απογόνους (ή κληρονόμους) που κληρονομούν τα κοινά χαρακτηριστικά, που βρίσκονται από πάνω τους.



Μ.Ι.Σ. 2025-2026

Αυτό στην αντικειμενοστρέφεια ονομάζεται κληρονομικότητα και αποτελεί ίσως την σπουδαιότερη τεχνική της ευκολία.

Η κλάση ΚΟΥΤΙΑ, είναι μια υπέρ-κλάση και οι κλάσεις ΟΡΘΟΓΩΝΙΑ ΚΟΥΤΙΑ και ΚΥΛΙΝΔΡΙΚΑ ΚΟΥΤΙΑ είναι δυο άλλες υπό-κλάσεις οι οποίες κληρονομούν τα κοινά χαρακτηριστικά της υπέρ-κλάσης.

Συνεπώς ένα καλογραμμένο πρόγραμμα διασπάται από τον κατασκευαστή κατά το στάδιο της σχεδίασης, σε επιμέρους κλάσεις με στόχο να απλοποιείται η ανάπτυξη του στην συνέχεια και να κατασκευάζονται ρουτίνες, συναρτήσεις, μέθοδοι μια φορά χωρίς άχρηστες επαναλήψεις.

Η κληρονομικότητα στην αντικειμενοστρέφεια εμφανίζεται σε απλή μορφή αλλά και σε πολλαπλή μορφή. Την κληρονομικότητα μπορούμε να την συναντήσουμε και στον ακολουθιακό προγραμματισμό και συνεπώς δεν θα πρέπει να θεωρείται αποκλειστικότητα του Ο.Ο.Ρ.

Υπό-Κλάσεις – Subclass

Η έννοια της υπό-κλάσης (subclass), περιγράφηκε στις προηγούμενες ενότητες και είναι μια κατάταξη αντικειμένων τα οποία δανείζονται ή κληρονομούν περιεχόμενο από μια ανώτερη κλάση. Η γνώση της έννοιας, θα μπορούσε να εμπλουτιστεί και με το ότι, οι κλάσεις δεν είναι απαραίτητο να φιλοξενούν μόνο δεδομένα (ύψος, περιγραφή, κ.λπ.) αλλά θα μπορούσαν να αφορούν καθαρό κώδικα.

Η σχέση δηλαδή της γενίκευσης – εξειδίκευσης, μπορεί να αφορά και μια υπό-ρουτίνα η οποία κληρονομεί χαρακτηριστικά λειτουργικότητας από μια ανώτερη ρουτίνα.

Πακέτα - Packages

Δημιουργώντας μεγάλους αριθμούς κλάσεων σε μια εφαρμογή είναι λογικό να οδηγηθούμε σε μεγάλο όγκο εντολών τις οποίες (για λόγους δομημένης σχεδίασης) δεν θα επιθυμούμε να αποθηκεύουμε σε ένα τεράστιο φυσικό αρχείο εντολών. Μπορούμε να διασπάμε τον κώδικα (τις κλάσεις) σε κομμάτια κώδικα. Τα κομμάτια αυτά μπορούν να «επικοινωνούν» μεταξύ τους, εάν δηλώσουμε ότι ανήκουν στο ίδιο πακέτο/package, (στην ίδια εφαρμογή δηλαδή).

Μ.Σ. 2025-2026

Ενθυλάκωση – Encapsulation

Με τον όρο ενθυλάκωση (encapsulation), στην αντικειμενοστρέφεια περιγράφουμε μια κατάσταση αφαιρετικότητας, η οποία έχει σκοπό να αποκρύπτει λεπτομέρειες από το να είναι ορατές, έτσι ώστε να απλοποιεί την παρακολούθηση ενός συστατικού λογισμικού. Κάποια δηλαδή από τα στοιχεία της κλάσης βρίσκονται νοητά καλυμμένα, ενθυλακωμένα, από μια περιβάλλουσα «μεμβράνη» απόκρυψης.
Η τεχνική στην ελληνική βιβλιογραφία παρουσιάζεται και με τον όρο κελυφοποίηση, του κώδικα σε επίπεδα θέασης.

Η κλάση δηλαδή αποκρύπτει τα στοιχεία της και δεν χρειάζεται να είναι ορατή και η παραμικρή της λεπτομέρεια.

Αρκεί το συστατικό λογισμικό να είναι λειτουργικό από μια κατάλληλη διεπαφή και να επιτελεί σωστά μια διεργασία.

Για παράδειγμα, εάν στην κλάση ΜΑΘΗΜΑΤΑ, θέλαμε να κατασκευάσουμε μια μέθοδο την οποία όταν θα την καλούμε θα αλλάζει την ώρα του μαθήματος κατά +1 ώρα, η κλήση και μόνο της ρουτίνας με όρισμα εισόδου, το όνομα του μαθήματος είναι αρκετό για να αλλάξει την ώρα, χωρίς να μας απασχολήσει πως είναι φτιαγμένη η κλάση ΜΑΘΗΜΑΤΑ, τι μεταβλητές περιλαμβάνει και πως η μέθοδος υπολογίζει και αλλάζει την ώρα.

Αυτές τις δυνατότητες τοποθέτησης μιας «μεμβράνης» απόκρυψης οι γλώσσες αντικειμενοστρέφειας το επιτυγχάνουν με κάποιες ειδικές δηλώσεις, ειδικότερα,

- τα στοιχεία μιας κλάσης (δεδομένα ή μέθοδοι) μπορούν να δηλωθούν ως **public**, **private** και **protected**, όπου αντίστοιχα θα σημαίνουν για όλα τα κομμάτια κώδικα που ανήκουν στο ίδιο πακέτο/package.

Μ.Ι.Σ. 2025-2026

Public Οι μεταβλητές, οι μέθοδοι και οι κλάσεις που δηλώνονται ως **public** είναι δυνατόν να είναι ορατά/ες από οποιαδήποτε άλλη κλάση όπου και αν βρίσκεται αυτή (σε οποιοδήποτε κομμάτι του πακέτου).

Private Όσες μεταβλητές, μέθοδοι και κλάσεις χαρακτηρίζονται ως **private** μπορούν να προσπελαστούν και να χρησιμοποιηθούν μόνο από το ίδιο το αντικείμενο (μέθοδο ή κλάση). Δεν μπορούν να προσπελαστούν από πουθενά έξω από αυτό.

Protected Οι μεταβλητές, οι μέθοδοι και οι κλάσεις που έχουν δηλωθεί ως **protected** σε μία κλάση μπορούν να προσπελαστούν από όλες τις κλάσεις που βρίσκονται στο ίδιο πακέτο ή από τις υποκλάσεις αυτής της κλάσης.

Θα είναι "ορατό"	**Public**	**Protected**	**χωρίς δήλωση**	**private**
Από την ίδια κλάση	✓	✓	✓	✓
Από τις κλάσεις του ίδιου πακέτου	✓	✓	✓	-
Από οποιοδήποτε κλάση εκτός του πακέτου όπου ανήκει η κλάση	✓	-	-	-
Από μία υποκλάση του ίδιου πακέτου	✓	✓	✓	-
Από μία υποκλάση έξω από το πακέτο όπου ανήκει η κλάση	✓	✓	-	-

Το θέμα της ενθυλάκωσης (encapsulation), μπορούμε να το συναντήσουμε σε πολλές πλατφόρμες αντικειμενοστρέφειας και στις περιπτώσεις αυτές μπορεί να παρουσιάζεται με πρόσθετες μορφές. Ο προσδιορισμός αυτής της ορατότητας μπορεί να δηλώνεται, με διαφορετικό τρόπο ανά γλώσσα αλλά γενικά ορίζονται τα επόμενα επίπεδα ορατότητας (εμβέλειας).

Δημόσια: στοιχείο προσβάσιμο από όλους τους κώδικες σε όλο το πακέτο.

Ιδιωτικά: στοιχείο προσβάσιμο μόνο στην κλάση που το ορίζει.

Προστατευόμενα: στοιχείο προσβάσιμο μόνο στην κλάση που το ορίζει και τις υποκλάσεις της.

Εσωτερικά: στοιχείο προσβάσιμο από όλον τον κώδικα, άλλα κανενός άλλου κώδικα στο πακέτο.

Προστατευόμενα εσωτερικά: στοιχείο προσβάσιμο από όλους τους κώδικες και των υποκλάσεων της κλάσης, σε όλο το πακέτο.

Ιδιωτικά προστατευόμενα: στοιχείο προσβάσιμο μόνο στην κλάση που το ορίζει και τις υποκλάσεις της μέσα στον κώδικα.

Μ.Ι.Σ. 2025-2026

Αφαιρετικότητα – Abstraction

Η αφαιρετικότητα (abstraction) αφορά την προσέγγιση μιας κλάσης μέσω μιας συνάρτησης η οποία χρησιμοποιεί μέρος των χαρακτηριστικών της κλάσης και αγνοεί όλα τα υπόλοιπα στοιχεία της.

Για παράδειγμα μια μέθοδος της κλάσης ΚΟΥΤΙΑ, μπορεί να αναζητά το υλικό του κουτιού, αγνοώντας παντελώς την ύπαρξη όλων των άλλων χαρακτηριστικών της κλάσης (περιγραφή, χρώμα, ύψος, κ.λπ.).

Υπερφόρτωση – Overloading

Με τον όρο υπερφόρτωση (overloading), στην αντικειμενοστρέφεια περιγράφεται η δυνατότητα να δημιουργούνται ρουτίνες (μέθοδοι) οι οποίες ενώ έχουν το ίδιο όνομα, περιέχουν διαφορετικό κώδικα και παράγουν διαφορετικά λειτουργικά αποτελέσματα. Αυτό επιτυγχάνεται με την διαφοροποίηση στα ορίσματα εισόδου.

Μια ρουτίνα δηλαδή, αναπτύσσεται με το ίδιο όνομα πολλές φορές αλλά με διαφορετικό σώμα εντολών και σε κάθε μια αυτές τις εκδόσεις τα ορίσματα εισόδου διαφέρουν. Η διαφοροποίηση μπορεί να αφορά το πλήθος αλλά και τους τύπους των ορισμάτων.

Αυτή η διαφοροποίηση, στα ορίσματα εισόδου, επιτρέπει στην γλώσσα να αναγνωρίζει την κατάλληλη έκδοση από τις συνώνυμες μεθόδους που θα πρέπει να ενεργοποιήσει.

Μ.Ι.Σ. 2025-2026

Πολυμορφισμός – Polymorphism

Με τον όρο πολυμορφισμός (polymorphism), η αντικειμενοστρέφεια περιγράφει την δυνατότητα, μεταξύ κλάσεων οι οποίες βασίζονται στην σχέση γενίκευσης – εξειδίκευσης, να δημιουργούνται ρουτίνες (μέθοδοι), οι οποίες να έχουν το ίδιο όνομα, να περιέχουν διαφορετικό κώδικα, να παράγουν διαφορετικά λειτουργικά αποτελέσματα, επενεργώντας επάνω σε διαφορετικά αντικείμενα.

Αυτό η γλώσσα το επιτυγχάνει βασιζόμενη στον τύπο του τρέχοντος αντικειμένου για το οποίο καλείται η συνώνυμη μέθοδος. Εάν δηλαδή το αντικείμενο είναι συμβατό με την υπέρ-κλάση καλείται η μέθοδος από εκεί, ενώ εάν το αντικείμενο είναι συμβατό με την υπό-κλάση καλείται η μέθοδος της υποκλάσης και ας έχουν το ίδιο όνομα. Η συνάρτηση δηλαδή καταλαβαίνει τι πρέπει να κάνει, βλέποντας τον τύπο του αντικειμένου για το οποίο καλείται και από εκεί και η εξήγηση του όρου πολύ-μορφισμός.

Μ.Ι.Σ. 2025-2026